Browser & Network DLP — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Browser & Network DLP in Microsoft Purview 5
The problem . . . 5
The solution . . . 5
How it works — the model . . . 5
What you get . . . 5
Scope and boundaries . . . 6
The bottom line . . . 6
1. Solution Overview — Browser & Network DLP (Microsoft Purview) 6
What this solution is . . . 6
Why this matters . . . 7
How it works . . . 7
What you can control . . . 8
Limitations (be honest with buyers) . . . 8
Naming convention (used in every doc and script) . . . 9
The standard deployment model . . . 9
2. Requirements Guide — Browser & Network DLP (Microsoft Purview) 10
2.1 Endpoint DLP licensing (browser & network DLP is plan-gated) . . . 10
2.2 Least-privilege DLP admin access (not blanket Global Admin) . . . 10
2.3 Security & Compliance PowerShell + Microsoft Graph toolchain . . . 10
2.4 Graph permission scopes (read-only for assessment) . . . 10
2.5 Devices onboarded to Microsoft Purview (Endpoint DLP) . . . 11
2.6 Microsoft Purview browser extension + browser coverage . . . 11
2.7 In-scope sensitive data (labels / sensitive information types) . . . 11
2.8 Destination inventory (unsanctioned & personal cloud domains) . . . 12
2.9 A pilot ring, simulation mode, and change control . . . 12
Checklist . . . 12
Global Requirements — Microsoft Purview Solutions 12
1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 13
2. Purview roles (least-privilege) and portal access . . . 13
3. Security & Compliance PowerShell + Graph toolchain . . . 13
4. Data sources & scope in scope . . . 14
5. Compliance, records, privacy & legal stakeholders . . . 14
6. A pilot scope and change control . . . 14
3. Implementation Guide — Browser & Network DLP (Microsoft Purview) 15
Step 1 — Assess the tenant (Assess) . . . 15

Step 2 — Design the in-scope data, destinations, and action ladder (Design) . . . 15
Step 3 — Confirm onboarding and browser coverage (Configure) . . . 16
Step 4 — Build the sensitive service domain group + browser restrictions (Configure) . . . 16
Step 5 — Author the DLP policy in simulation/test mode (Configure, pilot-first) . . . 16
Step 6 — Pilot in simulation and tune (Pilot → simulate) . . . 17
Step 7 — Validate on the pilot ring (Validate) . . . 17
Step 8 — Expand to enforcement (Expand → enforce) . . . 18
5. Testing Guide — Browser & Network DLP (Microsoft Purview) 18
Test 1 — Endpoint DLP is licensed and available . . . 18
Test 2 — Devices are onboarded and browsers are covered . . . 18
Test 3 — Policy dry-run makes no changes . . . 19
Test 4 — Policy is created in simulation/test mode (not enforcing) . . . 19
Test 5 — Sensitive upload to a restricted domain is detected in simulation . . . 19
Test 6 — Non-sensitive upload and sanctioned destination do not match . . . 19
Test 7 — Unallowed browser control steers correctly . . . 19
Test 8 — Evidence is captured in the audit log . . . 19
Test 9 — Enforcement transition is controlled . . . 19
Pass/fail gate . . . 20
Troubleshooting — Browser & Network DLP (Microsoft Purview) 20
1. A device shows no upload/egress activity at all . . . 20
2. Uploads are caught in Edge but Chrome/Firefox are invisible . . . 21
3. The rule fires on the wrong things (false positives) . . . 21
4. The rule doesn't fire on an upload it should . . . 21
5. Simulation looks fine but enforcement doesn't block . . . 22
6. `New-DlpComplianceRule` fails on the endpoint upload restriction . . . 22
7. `-EndpointDlpLocation` / scope errors . . . 23
8. Upload-activity report is empty or export fails . . . 23
9. `Get-PolicyConfig` can't read the domain group / unallowed browsers . . . 23
10. "It blocked an admin / broke a workflow" — emergency rollback . . . 24
When to escalate to the determination owners (not a technical fix) . . . 24
Best Practices — Browser & Network DLP (Microsoft Purview) 24
Deploy the way the house model says — simulate, then enforce . . . 24
Key the rule on *sensitive* content, not every upload . . . 25
Get the browser & domain settings right — they are the vocabulary . . . 25
Onboarding & extension hygiene . . . 25
Operate it, don't set-and-forget . . . 26

Keep the roles and determinations where they belong . . . 26
Security & handling of the tooling . . . 26
Deliverables Summary — Browser & Network DLP in Microsoft Purview 27
Documents . . . 27
PowerShell tools (5) . . . 28
How it maps to the deployment model . . . 29
Definition of done . . . 29
Appendix A — PowerShell Scripts 30

Executive Summary — Browser & Network DLP in Microsoft Purview

The problem

Users move sensitive data off the organization every day through the most ordinary path of all: the **web browser**. A document is dragged into a personal Google Drive or Dropbox, a spreadsheet is pasted into a consumer AI chat, a file is uploaded through a browser to an unsanctioned SaaS app. To the network this is just HTTPS; to compliance it is **data walking out the door with no control and no record**. Email and SharePoint DLP never see it — it happens in the browser, on the device. Left unaddressed, that is both a **compliance gap** and a direct **data-risk exposure**.

The solution

This solution delivers **Microsoft Purview Browser & Network DLP** end to end. It confirms devices are **onboarded** to Purview, deploys the **Microsoft Purview browser extension** (Chrome/Firefox) and relies on **native Edge** integration so browser uploads become visible, groups the destinations that matter — personal cloud and unsanctioned file-sharing/SaaS — into a **sensitive service domain group**, sets the **unallowed-browsers** control so sensitive items can't be moved through a browser Purview can't inspect, and authors a **Devices-scoped DLP policy** restricting uploads of your sensitive content (sensitivity labels and/or sensitive information types) to those destinations. Crucially, the control is **piloted in simulation/test mode first** — it records what it *would* audit, warn, or block without stopping anyone — validated on a small device ring, and only then moved to **enforce** (Audit → Warn → Block).

How it works — the model

The delivery follows a disciplined, repeatable model with an explicit pass/fail gate at each step:

Assess → **Design** → **Configure** → **Pilot (simulation)** → **Validate** → **Expand (enforce).**

Because a web-upload block stops legitimate work at the moment of the action, nothing is enforced until it has been proven, in simulation, to see the right browsers and destinations and act only on the right content — with a one-command rollback available at all times.

What you get

- A complete document set: Overview, Requirements, an ordered Implementation guide, Testing (pass/fail

gates), Troubleshooting, Best Practices, and this summary.

- **Five PowerShell tools** (Security & Compliance PowerShell / Microsoft Graph) — discovery,

configuration (simulation, `-WhatIf` first), validation, upload-activity reporting, and scope/health — all **read-only or preview by default, with no credentials in code.**

- A **defensibility evidence pack**: proof of what was configured, that the control was validated in

simulation before enforcement, and that enforcement behaves as designed.

Scope and boundaries

• Endpoint DLP browser & network controls are **licensing-dependent** (typically Microsoft 365 E5 / E5

Compliance / E5 Information Protection & Governance; some settings in preview) and the Purview portal **evolves** — capability availability and exact paths are **verified against your licensing/tenant**, never assumed.

• Coverage depends on **onboarding and the browser extension**; the unallowed-browsers control is a

steering control, not a guarantee against a determined user on unmanaged equipment.

• This solution focuses on **browser uploads to unsanctioned/personal web destinations**. Other Endpoint

DLP channels (USB/removable media, print, clipboard/screen capture, network share) and other DLP locations (Exchange, SharePoint, OneDrive, Teams, cloud apps via Defender for Cloud Apps) are the subject of their own dedicated solutions.

• **Regulatory, records, and legal determinations** — which data is sensitive, which destinations are

unsanctioned, whether an upload must be blocked versus audited, and whether an action is a violation — remain with your **compliance, security, records, privacy, and legal** functions. This solution provides the **technical method and the evidence**, not the determination.

The bottom line

The most common **uncontrolled web-upload path** for sensitive data becomes a **monitored, then enforced** control in Microsoft Purview — **configured, piloted in simulation, validated, and defensible.**

1. Solution Overview — Browser & Network DLP (Microsoft Purview)

What this solution is

Users upload sensitive data to the web every day — a document dragged into a personal Google Drive or Dropbox, a spreadsheet pasted into a consumer AI chat, a file sent through a browser to an unsanctioned SaaS app. From the network's point of view this is just HTTPS; from a compliance point of view it is data walking out the door with no control and no record. This solution closes that gap in **Microsoft Purview** by standing up **Browser & Network DLP** — Endpoint DLP's **browser and domain restrictions** enforced through the **Microsoft Purview browser extension** and native browser integration — so uploads to **unsanctioned and personal cloud services** are controlled, evidenced, and defensible.

Concretely, it delivers:

• **Device onboarding readiness** — confirm endpoints are onboarded to Purview (Endpoint DLP can only

see and control devices it is onboarded to).

• **Browser coverage** — deploy the **Microsoft Purview browser extension** (Chrome, Firefox) and use

the **native integration in Microsoft Edge**, and configure the **unallowed browsers** list so sensitive items can't be moved through a browser Purview can't inspect.

- **Sensitive service domains** — group the web destinations that matter (personal cloud, unsanctioned file-sharing/SaaS) into a **sensitive service domain group** and choose the action per activity — **Audit**, **Warn (block with override)**, or **Block** — for **uploading to a restricted domain**.
- **A Devices-scoped DLP policy** — conditioned on your sensitivity labels / sensitive information types, restricting the "upload to a restricted cloud service domain or access from an unallowed browser" activity — authored and **piloted in simulation/test mode first**, validated, then enforced.
- **Evidence** — endpoint egress activity surfaced in Activity Explorer and the unified audit log, so you can show what was blocked, warned, or audited.

Everything is done with the **Microsoft Purview portal** (purview.microsoft.com), **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** — no third-party tooling.

> ***Browser & network DLP is one lane of Endpoint DLP.*** *This solution focuses on* ***browser uploads to unsanctioned/personal web destinations*** *— the extension, unallowed browsers, sensitive service domains, and the upload restriction. Other Endpoint DLP activities (USB/removable media, print, clipboard/screen capture, copy-to-network-share) and other DLP locations (Exchange, SharePoint, OneDrive, Teams, cloud apps via Defender for Cloud Apps) are the subject of their own dedicated solutions. Where this guide reaches the edge of another lane, it points to that lane rather than duplicating it.*

Why this matters

- **Browser upload is the most common uncontrolled egress path.** Personal cloud and consumer SaaS are a click away in any browser; without browser DLP there is no policy and no record when sensitive data goes to them.
- **The browser is a blind spot unless Purview is inside it.** Endpoint DLP inspects browser activity through the **Purview extension** (Chrome/Firefox) and **native Edge** integration. A browser without that coverage is exactly why the **unallowed browsers** control exists — steer users to a browser Purview can see into, or block sensitive movement there.
- **Simulation turns a risky control into a safe rollout.** Blocking uploads tenant-wide on day one breaks legitimate work and generates a flood of tickets. Piloting in **simulation/test mode** shows exactly what *would* have been blocked — you tune first, enforce second.
- **"We control web uploads of sensitive data" needs evidence.** Auditors and regulators ask for the control *and* the proof. Activity Explorer and the audit log make the control defensible.

How it works

1. **Assess.** Read the tenant's compliance licensing (is Endpoint DLP available on the plan?), device

onboarding status, existing Devices-scoped DLP policies, and browser-extension readiness — read-only, no changes.

1. **Design.** Decide the **in-scope data** (which sensitivity labels / sensitive information types make an upload "sensitive"), the **destination groups** (personal cloud + unsanctioned SaaS), the **unallowed browsers** stance, and the **action ladder** (Audit → Warn → Block) per destination group.

1. **Configure.** Onboard/confirm devices, deploy the Purview browser extension, build the **sensitive service domain group **in Endpoint DLP settings, and author the** Devices-scoped DLP policy **— the policy is created in** simulation/test mode**, not enforcing.

1. **Pilot (simulate).** Run the policy in **simulation / test mode** on a pilot ring of devices/users; review what it *would* block or warn on in Activity Explorer, and tune the domain group, conditions, and actions.

1. **Validate.** Confirm the extension is active, the domain group and unallowed-browser controls behave, simulation shows the expected matches with acceptable false positives, and every Testing-Guide gate passes on the ring.

1. **Expand (enforce).** Move the tuned policy from simulation to **enforce** (Warn, then Block where appropriate), broaden device scope, and keep the audit/Activity-Explorer evidence flowing.

What you can control

• Which **devices** are in scope (onboarded to Purview; a pilot ring first).

• Which **browsers** are covered (extension on Chrome/Firefox, native Edge) and which are **unallowed**.

• Which **web destinations** are restricted (the sensitive service domain group: personal cloud + unsanctioned SaaS).

• What makes an upload **sensitive** (sensitivity labels and/or sensitive information types as the DLP condition).

• The **action per activity/destination**: Audit, Warn (block with override + business justification), or Block.

• The **policy mode**: simulation/test first, enforce after validation.

Limitations (be honest with buyers)

• **Endpoint DLP is licensing-gated.** Browser & network DLP capabilities require the appropriate plan (**Microsoft 365 E5**, the **E5 Compliance** add-on, or **E5 Information Protection & Governance**) — **verify availability against your own tenant and plan.** Some settings are in preview.

• **Coverage depends on the extension and onboarding.** Chrome and Firefox require the **Microsoft Purview browser extension**; Edge uses native integration. A device that isn't** onboarded**, or a browser without coverage, is why the** unallowed browsers** control exists — it is a steering control, not a guarantee against a determined user on unmanaged equipment.

• **The portal is evolving.** Microsoft is actively changing the Purview portal and Endpoint DLP settings surfaces. Portal paths (e.g. *Data Loss Prevention → Endpoint DLP settings → Browser and domain restrictions to sensitive data*) are cited as of authoring and flagged **"verify in your tenant"** — never invented.

• **Some Endpoint DLP settings are portal-managed.** The **sensitive service domain groups**, the **unallowed browsers** list, and extension deployment are configured in the **portal / management tooling**; the provided scripts assess readiness, author and validate the **DLP policy** (which *does* have a supported PowerShell surface), and report activity — they do not invent cmdlets for settings Microsoft manages in the portal.

• **Determinations are deferred.** Which data is sensitive enough to restrict, which destinations are "unsanctioned," and any regulatory/records/legal obligation are decided by the organization's compliance, security, privacy, and legal functions — this solution provides the technical method and evidence, not the determination.

Naming convention (used in every doc and script)

Artifacts follow `<Purpose> – Purview – <Stage>` and are reused verbatim across docs, scripts, and the portal.

Artifact	Name
Readiness / egress assessment export	`Browser & Network DLP – Egress – Assess`
Sensitive service domain group	`Unsanctioned & Personal Cloud – Domain Group`
DLP policy (Devices)	`Browser & Network DLP – Devices – Pilot`
DLP rule (upload restriction)	`Restrict Upload to Unsanctioned Domains – Rule`
Validation snapshot	`Browser & Network DLP – Endpoint – Validation`
Egress activity report	`Browser & Network DLP – Egress – Report`
Device / extension health map	`Browser & Network DLP – Device Health – Map`
Pilot ring	`Browser & Network DLP Pilot Ring – Stage1`

The standard deployment model

> *Assess the tenant → design the in-scope data, destinations, and action ladder → configure onboarding, extension, domain group, and the policy in simulation → pilot on a ring and tune → validate every gate → expand to enforce. Pattern:* ***Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce).***

2. Requirements Guide — Browser & Network DLP (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance licensing, least-privilege Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

2.1 Endpoint DLP licensing (browser & network DLP is plan-gated)

Description: A confirmed compliance plan that includes **Endpoint DLP** — **Microsoft 365 E5**, the **E5 Compliance** add-on, or **E5 Information Protection & Governance**. Browser and domain restrictions are an Endpoint DLP capability (Global Requirement 1). **Why:** Without the right plan the **Devices** location and the browser/domain restriction settings are not available; the policy cannot be enforced. **Verify:** M365 admin center → `Billing → Licenses` shows the E5/compliance SKU; run `DLPBROWSER-Discovery.ps1` — it reads `subscribedSkus` via Graph and flags whether an Endpoint DLP–bearing SKU is present. Confirm **Data Loss Prevention → Endpoint DLP settings** is visible in the Purview portal (verify path). **Implement:** Assign/purchase the required licensing before configuring. Where it is not licensed, record browser & network DLP as out-of-scope rather than attempting to enable it.

2.2 Least-privilege DLP admin access (not blanket Global Admin)

Description: An account in **Compliance Administrator** or **Compliance Data Administrator** (to author DLP policies and Endpoint DLP settings), per Global Requirement 2. **DLP Compliance Management** role is included in those groups. **Why:** DLP policy authoring and Endpoint DLP settings require the matching Purview role; over-granting Global Administrator undermines least privilege. **Verify:** Purview portal → `Settings → Roles & scopes → Role groups` (verify path) — confirm membership in Compliance Administrator / Compliance Data Administrator; `Connect-IPPSSession` succeeds and `Get-DlpCompliancePolicy` returns without an authorization error. **Implement:** Add the account to the least-privilege compliance role group under change control.

2.3 Security & Compliance PowerShell + Microsoft Graph toolchain

Description: A workstation with the **ExchangeOnlineManagement** module (`Connect-IPPSSession` for Security & Compliance PowerShell, which hosts the `*-DlpCompliancePolicy` / `*-DlpComplianceRule` and `Search-UnifiedAuditLog` cmdlets) and the **Microsoft Graph PowerShell SDK**, with a modern-auth/MFA admin account (Global Requirement 3). **Why:** Assessment, policy authoring/validation, and reporting run through these. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement, Microsoft.Graph.Authentication`; `Connect-IPPSSession` and `Connect-MgGraph` both succeed. `DLPBROWSER-Discovery.ps1` installs missing modules and reports connectivity. **Implement:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser` and `Install-Module Microsoft.Graph -Scope CurrentUser`; confirm modern-auth sign-in.

2.4 Graph permission scopes (read-only for assessment)

Description: Delegated Graph scopes for the read-only assessment/health scripts: `Organization.Read.All` (licensing), `Directory.Read.All`, and `DeviceManagementManagedDevices.Read.All` (managed-device / endpoint readiness signal). No write scopes are required. **Why:** The readiness and device-health scripts read licensing and managed-device data through Graph; keeping to read scopes matches the safe-by-default posture. **Verify:** Run `DLPBROWSER-ScopeHealth.ps1` (read-only) — it connects with the read scopes and reports if consent is missing. **Implement:** Consent at first `Connect-MgGraph`; if admin consent is required, a Global Administrator approves the scopes once in Entra (`Identity → Applications → Enterprise applications`).

2.5 Devices onboarded to Microsoft Purview (Endpoint DLP)

Description: The in-scope **Windows 10/11 and/or macOS endpoints onboarded to Purview** for Endpoint DLP. Onboarding is separate from device management/enrollment; a device must be onboarded before Endpoint DLP (and therefore browser DLP) can see or control it (Global Requirement 4). **Why:** Browser & network DLP is enforced on **onboarded devices** only — an un-onboarded device is invisible to the policy. **Verify:** Purview portal → `Settings → Device onboarding` (verify path) shows onboarded devices; run `DLPBROWSER-ScopeHealth.ps1` — it reports managed-device counts and onboarding readiness signals. **Implement:** Onboard the pilot-ring devices first (portal → Device onboarding; script/GPO/Intune per Microsoft's onboarding method). Roll out broadly only after the pilot validates.

2.6 Microsoft Purview browser extension + browser coverage

Description: The **Microsoft Purview browser extension** deployed to **Chrome** and **Firefox** on in-scope devices, with **Microsoft Edge** relying on its **native** Endpoint DLP integration (no extension needed). This is what lets Purview inspect browser upload activity. **Why:** Chrome/Firefox upload activity is only visible to Endpoint DLP when the extension is present; without it those browsers are effectively blind and should be treated via the **unallowed browsers** control. **Verify:** On a pilot device, confirm the extension is installed and enabled in Chrome/Firefox (`chrome://extensions` / Firefox add-ons); `DLPBROWSER-ScopeHealth.ps1` reports the extension as a readiness checklist item (deployment is enterprise-managed, verify in your MDM/GPO). **Implement:** Deploy the extension via **Group Policy / Intune / enterprise browser management** per Microsoft's guidance (Purview portal → Endpoint DLP settings references the extension). Verify the current extension name/ID in your tenant — the portal is evolving.

2.7 In-scope sensitive data (labels / sensitive information types)

Description: The **sensitivity labels** and/or **sensitive information types (SITs)** that define when an upload is "sensitive" and therefore in scope for the policy condition (Global Requirement 4). **Why:** The DLP rule keys off these; without a defined condition the policy either matches nothing or matches everything. **Verify:** Purview portal → `Information Protection → Sensitivity labels` and `Data classification → Sensitive info types` show the intended labels/SITs; confirm with stakeholders which represent restrict-worthy data. **Implement:** Reuse the label taxonomy / SITs from the Information Protection solutions; where none exist yet, start with high-confidence built-in SITs (e.g. credit-card / national-ID types) for the pilot and refine. **Which data warrants restriction is a compliance/security determination**, not a technical default.

2.8 Destination inventory (unsanctioned & personal cloud domains)

Description: The list of **web destinations to restrict** — personal cloud storage and unsanctioned file-sharing/SaaS domains — that will populate the **sensitive service domain group** `Unsanctioned & Personal Cloud – Domain Group`. **Why:** The domain group is what the upload restriction acts on; an incomplete list leaves egress paths uncontrolled. **Verify:** Review egress signals (Activity Explorer, and any Defender for Cloud Apps / proxy discovery) to see where uploads actually go; `DLPBROWSER-Report.ps1` summarizes recent endpoint egress from the audit log to inform the list. **Implement:** Compile the domain list with security/stakeholders; enter it as a sensitive service domain group in Purview portal → **Endpoint DLP settings** → **Browser and domain restrictions to sensitive data** (verify path). **Which destinations are "unsanctioned" is an organizational determination.**

2.9 A pilot ring, simulation mode, and change control

Description: A small **pilot ring** (`Browser & Network DLP Pilot Ring – Stage1`) of devices/users, confirmation that the DLP policy can run in **simulation / test mode** before enforcement, and a **change-control** process, per Global Requirement 6. **Why:** Browser upload blocking is high-impact; the policy must be **piloted in simulation/test** and tuned before any Warn/Block enforcement. **Verify:** The ring is defined; `New-DlpCompliancePolicy`/`Set-DlpCompliancePolicy` support the test modes (`TestWithoutNotifications` / `TestWithNotifications`); a change window/approval path exists. **Implement:** Define the ring and agree change control before enforcing anything.

Checklist

- Endpoint DLP licensing confirmed (E5 / E5 Compliance / E5 IP&G) — else recorded out-of-scope
- Compliance Administrator / Compliance Data Administrator access confirmed; Global Admin not required for authoring
- ExchangeOnlineManagement + Microsoft Graph SDK installed; `Connect-IPPSSession` / `Connect-MgGraph` succeed
- Graph read scopes consented (no write scopes needed)
- In-scope devices onboarded to Purview (pilot ring first)
- Microsoft Purview browser extension deployed (Chrome/Firefox); Edge native coverage confirmed
- In-scope sensitivity labels / SITs identified as the policy condition
- Unsanctioned & personal-cloud destination list compiled for the domain group
- `Browser & Network DLP Pilot Ring – Stage1` defined; simulation/test mode confirmed; change control agreed

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a

solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.

- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.
- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.
- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).
- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.
- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.
- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.
- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.
- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant

governance functions.*

3. Implementation Guide — Browser & Network DLP (Microsoft Purview)

Follow these steps **in order**. Work is done in the **Microsoft Purview portal** (purview.microsoft.com), **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph**. The DLP policy is authored in **simulation/test mode** and previewed with `-WhatIf` before any `-Commit`/enforce. Artifact names follow the `<Purpose> – Purview – <Stage>` convention from the Overview and are reused verbatim.

> ***Model:*** *Assess the tenant → design the in-scope data, destinations, and action ladder → configure onboarding, extension, domain group, and the policy in simulation → pilot on a ring and tune → confirm every gate → expand to enforce. Pattern:* ***Assess*** *→* ***Design*** *→* ***Configure*** *→* ***Pilot (simulate)*** *→* ***Validate*** *→* ***Expand (enforce). Portal paths are cited as of authoring and the Purview portal / Endpoint DLP settings are evolving — verify each path in your own tenant.*** *Capability availability is licensing-dependent — verify against your plan (Requirement 2.1). The upload restriction is* ***high-impact****: it is piloted in* ***simulation/test*** *and tuned before Warn/Block enforcement.*

Step 1 — Assess the tenant (Assess)

1. Run `DLPBROWSER-Discovery.ps1 -CsvPath .\DlpBrowserReadiness.csv`. Read-only, it captures:

whether an **Endpoint DLP–bearing SKU** is present (E5 / E5 Compliance / E5 IP&G), existing **Devices-scoped DLP policies** and their **mode** (enabled vs test/simulation), and S&C PowerShell + Graph connectivity. It writes `Browser & Network DLP – Egress – Assess`.

1. Run `DLPBROWSER-ScopeHealth.ps1 -CsvPath .\EndpointHealth.csv` to record **device onboarding**

readiness and the **browser-extension** checklist for the pilot ring.

1. **Pass/fail:** you can state that Endpoint DLP is available on the plan, how many devices are

onboard-ready, and whether any Devices DLP policy already exists (to avoid conflicts).

Step 2 — Design the in-scope data, destinations, and action ladder (Design)

1. Confirm the **policy condition**: the sensitivity labels and/or sensitive information types that make

an upload "sensitive" (Requirement 2.7). Record them.

1. Compile the **destination list** for `Unsanctioned & Personal Cloud – Domain Group` — personal cloud

storage and unsanctioned file-sharing/SaaS domains (Requirement 2.8). Use `DLPBROWSER-Report.ps1` to see where uploads actually go today.

1. Decide the **unallowed browsers** stance (which browsers to steer users away from where the extension

isn't present) and the **action ladder** per destination group: **Audit** first (simulation), then **Warn** (block with override + business justification), then **Block** for the highest-risk destinations. **Which data and which destinations warrant restriction are compliance/security determinations** — you implement them, stakeholders decide.

1. **Pass/fail:** a one-page design names the condition (labels/SITs), the domain group members, the unallowed-browser stance, and the action per group.

Step 3 — Confirm onboarding and browser coverage (Configure)

1. Ensure the pilot-ring devices are **onboarded to Purview**: Purview portal → `Settings → Device onboarding `(verify path). Re-run` DLPBROWSER-ScopeHealth.ps1` until the ring reports onboarded.

1. Deploy the **Microsoft Purview browser extension** to **Chrome** and **Firefox** on the ring (Group Policy / Intune / enterprise browser management, per Microsoft's guidance). **Microsoft Edge** uses its **native** integration — no extension. Verify the extension is enabled on a pilot device.

1. **Pass/fail:** every pilot device is onboarded and shows the extension enabled in Chrome/Firefox; Edge is confirmed as natively covered.

Step 4 — Build the sensitive service domain group + browser restrictions (Configure)

1. In Purview portal → **Data Loss Prevention → Endpoint DLP settings → Browser and domain restrictions to sensitive data** (verify path):

• Create the **sensitive service domain group** `Unsanctioned & Personal Cloud – Domain Group` and add the destinations from Step 2. Set the per-activity action for **"Upload to a restricted cloud service domain"** to **Audit** for the pilot (you will escalate to Warn/Block after validation).

• Configure the **unallowed browsers** list per the Step 2 stance.

1. These Endpoint DLP **settings** are portal-managed — record the exact group name and action so the DLP policy (next step) and the docs stay consistent. Do not attempt to script settings Microsoft manages in the portal.

1. **Pass/fail:** the domain group exists with the intended members and a starting action of **Audit**; the unallowed-browsers list matches the design.

Step 5 — Author the DLP policy in simulation/test mode (Configure, pilot-first)

1. Dry-run first: `DLPBROWSER-Configure.ps1 -WhatIf`. With `-WhatIf` (the default) it makes **no

changes **— it prints the exact** `New-DlpCompliancePolicy` **/** `New-DlpComplianceRule` **it *would* run, including the policy name** `Browser & Network DLP – Devices – Pilot`**, the** Devices **location scoped to the pilot ring, the rule** `Restrict Upload to Unsanctioned Domains – Rule` **conditioned on your labels/SITs, and the** test mode**.

1. Create in simulation: `DLPBROWSER-Configure.ps1 -Commit -Mode TestWithoutNotifications

-Sits '<SIT names>' (or -SensitivityLabels '<label GUIDs>'`). The policy is created **in test mode** — it evaluates and records matches but **does not block or notify** users. The endpoint upload restriction is realized through the domain group's action (Step 4) once the policy scopes it; keep the domain-group action at **Audit** during simulation.

1. Scope the policy to `Browser & Network DLP Pilot Ring – Stage1` (a device group / user set), not the

whole tenant.

1. **Pass/fail + rollback:** `Get-DlpCompliancePolicy -Identity 'Browser & Network DLP – Devices – Pilot'`

shows `Mode = TestWithoutNotifications` and the Devices location. Rollback is `Set-DlpCompliancePolicy -Identity '<name>' -Mode Disable` or `Remove-DlpCompliancePolicy` (the script prints the exact reverse command).

Step 6 — Pilot in simulation and tune (Pilot → simulate)

1. Let the policy run in **test/simulation** on the ring for a representative period. From a pilot device,

attempt an upload of sensitive test content to a domain in the group.

1. Review what the policy **would have** acted on: Purview portal → the policy's **simulation/overview**

(verify path) and `Data classification → Activity explorer`; run `DLPBROWSER-Validate.ps1` to confirm the policy mode, rules, and match counts, and `DLPBROWSER-Report.ps1` to summarize endpoint egress events.

1. **Tune:** adjust the domain group membership, the labels/SITs condition, and the action ladder to

remove false positives and cover missed destinations. Re-run until the simulation matches the intent.

1. **Pass/fail:** simulation shows the expected uploads matched with an acceptable false-positive rate;

the Testing Guide gates pass on the ring.

Step 7 — Validate on the pilot ring (Validate)

1. Complete the **Testing Guide** on `Browser & Network DLP Pilot Ring – Stage1`: extension active,

onboarded devices covered, domain group behaving, simulation producing the expected matches, and evidence appearing in Activity Explorer / the audit log.

1. Confirm the **unallowed browsers** control steers users as designed, and that a covered browser

(Edge / extension-equipped Chrome/Firefox) is what actually enforces.

1. **Pass/fail:** all Testing-Guide gates pass; stakeholders sign off on moving from simulation to

enforcement.

Step 8 — Expand to enforcement (Expand → enforce)

1. Escalate the action ladder: in Endpoint DLP settings, move the domain group's upload action from

Audit to **Warn (block with override)**, and to **Block** for the highest-risk destinations, per the approved design. Move the policy from test to enforce: `DLPBROWSER-Configure.ps1 -Enforce` (wraps `Set-DlpCompliancePolicy -Mode Enable`) — dry-run `-WhatIf` first.

1. **Broaden device scope** from the pilot ring to the full onboarded set in stages, re-validating at

each ring.

1. Keep evidence flowing — `DLPBROWSER-Report.ps1` on a schedule — and hand off adjacent lanes

(USB/removable media, print, clipboard/screen capture, Exchange/SharePoint/OneDrive/Teams DLP, cloud apps via Defender for Cloud Apps) to their dedicated solutions.

Order of operations

`Assess tenant + endpoints → Design condition/destinations/action ladder → Confirm onboarding + browser extension → Build sensitive service domain group (Audit) + unallowed browsers → Author DLP policy in test/simulation (dry-run then -Commit) → Pilot in simulation and tune → Validate on ring → Escalate to Warn/Block + expand device scope (enforce)`

5. Testing Guide — Browser & Network DLP (Microsoft Purview)

Run after building the readiness assessment and after each configuration step. Every test is safe (read-only or a `-WhatIf` preview) except the deliberate **sensitive-upload attempt on a pilot device**, which is expected and controlled. Validate on `Browser & Network DLP Pilot Ring – Stage1` **in simulation/test mode** before enforcing anything tenant-wide.

Test 1 — Endpoint DLP is licensed and available

Do: Run `DLPBROWSER-Discovery.ps1`; compare against `Billing → Licenses` and confirm **Data Loss Prevention → Endpoint DLP settings** appears in the Purview portal (verify path). **Expected:** An Endpoint DLP–bearing SKU (E5 / E5 Compliance / E5 IP&G) is reported present, and the Endpoint DLP settings surface is visible — browser & network DLP is available on this plan.

Test 2 — Devices are onboarded and browsers are covered

Do: Run `DLPBROWSER-ScopeHealth.ps1`; on a pilot device confirm the Microsoft Purview browser extension is enabled in Chrome/Firefox and Edge native coverage applies. **Expected:** Pilot-ring devices report onboarded to Purview; the extension is active in Chrome/Firefox; Edge is confirmed natively covered. Any gap is recorded as a hand-off item, not silently ignored.

Test 3 — Policy dry-run makes no changes

Do: Run `DLPBROWSER-Configure.ps1 -WhatIf`; afterward run `Get-DlpCompliancePolicy -Identity 'Browser & Network DLP – Devices – Pilot'`. **Expected:** The dry-run prints the exact `New-DlpCompliancePolicy`/`New-DlpComplianceRule` it *would* run and the reverse command; the policy does **not** yet exist (or is unchanged) — `-WhatIf` changed nothing.

Test 4 — Policy is created in simulation/test mode (not enforcing)

Do: After `-Commit -Mode TestWithoutNotifications`, run `DLPBROWSER-Validate.ps1`. **Expected:** The policy exists with `Mode = TestWithoutNotifications`, scoped to the **Devices** location and the pilot ring, with the rule conditioned on the intended labels/SITs — it is evaluating, **not blocking or notifying** users.

Test 5 — Sensitive upload to a restricted domain is detected in simulation

Do: From a pilot device, upload sensitive **test** content (matching the policy condition) to a domain in `Unsanctioned & Personal Cloud – Domain Group` through a covered browser. **Expected:** In simulation the upload is **allowed to proceed** (test mode doesn't block) but is **recorded** — it surfaces in `Data classification → Activity explorer` and via `DLPBROWSER-Report.ps1` as a would-be match against the domain group.

Test 6 — Non-sensitive upload and sanctioned destination do not match

Do: From a pilot device, upload **non-sensitive** content, and upload sensitive test content to a **sanctioned** destination not in the domain group. **Expected:** Neither produces a policy match — the condition (labels/SITs) and the destination scope (domain group) are working, so the policy isn't over-matching.

Test 7 — Unallowed browser control steers correctly

Do: On a pilot device, attempt the sensitive upload from a browser designated **unallowed** (no extension coverage). **Expected:** The unallowed-browser control behaves per the design (access to sensitive items steered/ prevented in that browser), confirming coverage comes only from Edge / extension-equipped browsers.

Test 8 — Evidence is captured in the audit log

Do: Run `DLPBROWSER-Report.ps1` over the test window; cross-check the Purview audit search (`Audit → Search`, verify path). **Expected:** `Browser & Network DLP – Egress – Report` lists the endpoint egress events (would-be blocks/warns/audits) with device, destination, and action — the control is evidenced and defensible.

Test 9 — Enforcement transition is controlled

Do: Only after Tests 1–8 pass on the ring: run `DLPBROWSER-Configure.ps1 -Enforce -WhatIf`, then `-Enforce`; escalate the domain group's action from Audit to Warn/Block per design. **Expected:** The policy moves to `Mode = Enable`; a subsequent sensitive upload is **warned (override + justification)** or **blocked** per the destination's action — enforcement matches what simulation predicted, with no surprises.

Pass/fail gate

Gate	Requirement
Simulation pass	Tests 1–8 pass on `Browser & Network DLP Pilot Ring – Stage1` in **test/simulation** mode: licensed, devices onboarded + browsers covered, policy created in test mode, sensitive uploads detected (not over-matching), unallowed-browser steering works, evidence captured
Enforcement pass	Test 9 passes: after stakeholder sign-off, the policy moves to enforce and the escalated Warn/Block actions behave exactly as simulation predicted, expanded ring-by-ring

Troubleshooting — Browser & Network DLP (Microsoft Purview)

The failures you will actually hit, with the most likely cause and the fix. Work top-down: **most "the upload isn't being caught" reports trace back to onboarding, the browser extension, condition matching, the sensitive service domain group, or simulation-vs-enforce confusion — not the rule itself.** Where a portal path or cmdlet parameter is named, **verify it against your tenant** — the Purview portal and the Endpoint DLP browser/domain settings evolve.

1. A device shows no upload/egress activity at all

Symptoms: Upload a matching file to a restricted domain on a device and **nothing** appears in Activity explorer (Devices); the policy never fires.

Likely causes & fixes

• **Device isn't onboarded.** The #1 cause. Confirm the device appears under **Purview** → **Devices**

(Settings → Device onboarding). If not, re-run your onboarding method (Intune/GPO/local script/ ConfigMgr). There is **no** S&C PowerShell cmdlet to onboard — it is a portal/management-tool action.

• **Browser not covered.** In Chrome/Firefox, no Microsoft Purview browser extension means no visibility

(see §2). In Edge, confirm native integration applies.

• **Onboarding still propagating.** After onboarding, allow time for the device to register and for

activity to surface — endpoint telemetry is **not** instant. Wait and retry.

• **Check the readiness signals:** `DLPBROWSER-ScopeHealth.ps1` reports managed-device counts (Graph),

the licensing flag, and the coverage checklist — but remember the device count is a **proxy**; the authoritative check is the **Devices** list in the portal.

2. Uploads are caught in Edge but Chrome/Firefox are invisible

Symptoms: Cloud-egress/upload is caught in **Edge** but not in **Chrome** or **Firefox**.

Fix

• **Deploy the Microsoft Purview browser extension** to Chrome/Firefox (Requirement 2.6). Edge is native; Chrome/Firefox **cannot** see upload/cloud-egress activity without the extension.

• Until the extension is deployed, treat those browsers as **unallowed browsers** in the Endpoint DLP settings so uploads through them are controlled rather than silently uncovered.

• Verify the extension is actually installed and enabled on a sample device (via your browser enterprise policy). `DLPBROWSER-ScopeHealth.ps1` lists this as a checklist item — deployment itself is enterprise-managed, not scriptable here.

3. The rule fires on the wrong things (false positives)

Symptoms: Legitimate work is flagged/blocked — an upload to a **sanctioned** destination, or non-sensitive content.

Likely causes & fixes

• **Sanctioned destination in the restricted group.** Remove corporate OneDrive/SharePoint and approved SaaS from `Unsanctioned & Personal Cloud – Domain Group`; that group should hold only the destinations you intend to control.

• **Condition too broad.** If the rule keys on "all content" instead of a **sensitivity label / SIT**, it will match everything. Tighten the condition (Requirement 2.7). `DLPBROWSER-Validate.ps1` gate **A6** flags a rule with no readable content condition.

• **SIT confidence/count too loose.** Raise the confidence level or instance count on the sensitive information type to cut noise, then re-test.

• Tune **in simulation** — never tune by loosening an already-enforcing block in production.

4. The rule doesn't fire on an upload it should

Symptoms: A file you expect to match is uploaded to a restricted domain with no policy match.

Likely causes & fixes

• **Content doesn't carry the label/SIT.** The test file must actually carry the **sensitivity label** or contain the **sensitive information type** the rule keys on. Seed a known match and confirm the label is applied / the SIT pattern is present.

• **Destination not in the domain group.** Confirm the target domain is a member of

`Unsanctioned & Personal Cloud – Domain Group` (a missing destination is uncontrolled). Use `DLPBROWSER-Report.ps1` to see where uploads actually go and refine the list.

- **Label not published / SIT not in scope.** Confirm the label is published to the user (Information Protection → Label policies) and the SIT exists (Data classification). `DLPBROWSER-Discovery.ps1` inventories both.
- **Rule disabled.** `DLPBROWSER-Validate.ps1` gate **A5b** flags a disabled rule.

5. Simulation looks fine but enforcement doesn't block

Symptoms: In simulation the upload is recorded; after "turning it on" the block/warn still doesn't happen.

Likely causes & fixes

- **Policy still in a Test mode.** Simulation records but does **not** block. To enforce, move the policy out of simulation: portal → turn the policy **On**; PowerShell → `Set-DlpCompliancePolicy -Identity 'Browser & Network DLP - Devices - Pilot' -Mode Enable`. `DLPBROWSER-Validate.ps1` gate **A3** shows the current mode.
- **Domain-group action still Audit.** The sensitive service domain group's action for "upload to a restricted domain" may still be **Audit** even though the policy is enforcing. Escalate it to **Warn/Block** in Endpoint DLP settings per the approved design.
- **Device out of scope.** Enforcement only applies to devices in the policy's Devices scope. Confirm with `DLPBROWSER-ScopeHealth.ps1` / gate **A4**.
- **Propagation delay.** Policy/action changes take time to reach endpoints. Wait, then retest.

6. `New-DlpComplianceRule` fails on the endpoint upload restriction

Symptoms: `DLPBROWSER-Configure.ps1 -Commit` (or manual scripting) errors when setting the endpoint upload restriction, e.g. an invalid Setting name or value.

Fix

- **Verify the exact Setting name and values for your tenant.** The endpoint-restriction Setting name for the upload/cloud-egress activity and its permitted values (`Audit` / `Warn` / `Block` / `Ignore`) **evolve**. Run `Get-Help New-DlpComplianceRule -Full` and check current Microsoft docs, then adjust the hashtable. `DLPBROWSER-Configure.ps1` deliberately **prints the planned Setting name** so you can verify before `-Commit`.
- **Remember the destination side is portal-managed.** The sensitive service domain group and the unallowed-browsers list are configured in **Endpoint DLP settings** (portal), not by the script — build them there first, then key the policy/rule to your content condition.
- **Prefer the portal for the first build** if PowerShell parameter names are uncertain, then read the result back with `DLPBROWSER-Discovery.ps1` to learn the exact shape in your tenant.

7. `-EndpointDlpLocation` / scope errors

Symptoms: Creating the policy errors on the location/scope value.

Fix

- The **Devices** location is scoped by user/group (or `All`). Confirm the pilot group

(`SG-DLPBROWSER-Pilot`) exists and the identifier is one your tenant accepts. `DLPBROWSER-ScopeHealth.ps1` reports the group and the current scope.

- Avoid `All` during the pilot — scope to the pilot group first (gate **A4**).

8. Upload-activity report is empty or export fails

Symptoms: `DLPBROWSER-Report.ps1` returns no upload activity, or the Activity Explorer export warns and skips.

Likely causes & fixes

- **Licensing/permission.** `Export-ActivityExplorerData` is licensing-dependent; the script **warns and

degrades to policy-modes only **rather than failing. Fall back to** Purview → Activity explorer **(filter to** Devices** + upload/egress activities) in the portal.

- **Keyword filter missed the activity-type name.** Endpoint activity-type names evolve; the script falls

back to exporting **all** rows for manual review when its upload/egress keywords match nothing — open the CSV and look for the upload/cloud-egress events directly.

- **Quiet pilot window.** A small pilot may simply have no activity in the window — widen `-Days` or

perform the sanctioned test upload from `05-testing.md` first.

- **Role.** Ensure the connected account holds a role that can read Activity Explorer.

9. `Get-PolicyConfig` can't read the domain group / unallowed browsers

Symptoms: `DLPBROWSER-Discovery.ps1` / `DLPBROWSER-Validate.ps1` report the sensitive service domain group or unallowed-browsers list as not readable (gate **A7 / A7b** = UNKNOWN).

Fix

- **This is expected on some tenants.** The Endpoint DLP browser/domain **settings** surface and the

`Get-PolicyConfig` property shape **evolve**, and these settings are **portal-managed**. UNKNOWN is not a failure — **confirm the domain group and unallowed-browsers list in the portal**: Data Loss Prevention → Endpoint DLP settings → **Browser and domain restrictions to sensitive data** (verify path).

- Do not attempt to script settings Microsoft manages in the portal — the scripts deliberately read and

report them, and configure only the policy/rule that has a supported PowerShell surface.

10. "It blocked an admin / broke a workflow" — emergency rollback

Do this immediately, then diagnose:

- Return the policy to **simulation** or **off**:

`Set-DlpCompliancePolicy -Identity 'Browser & Network DLP - Devices - Pilot' -Mode TestWithoutNotifications` (simulation) or `-Mode Disable` (off). Either **stops enforcement at once.**

- Or return the sensitive service domain group's upload action to **Audit** in Endpoint DLP settings.
- Then work the relevant section above (usually §3 false positives or §5 scope/action), fix **in simulation**, re-validate with `DLPBROWSER-Validate.ps1`, and only then re-enforce.

When to escalate to the determination owners (not a technical fix)

Some questions are **not** troubleshooting — they are **determinations** that belong to compliance / security / records / HR-legal (see `global-requirements-purview.md` §5):

- *Should* this destination be **blocked** or only **audited**? → the action-ladder decision owners.
- Is *this destination* actually "unsanctioned," and is *this content* in scope? → **compliance / data owners**.
- Is a user's upload a **violation**, and what is the consequence? → **HR / legal**.

Provide them the **evidence** (`DLPBROWSER-Report.ps1`, Activity explorer, the validation output) — not a verdict.

Best Practices — Browser & Network DLP (Microsoft Purview)

Field-tested guidance for running Browser & Network DLP well. It complements — does not replace — the implementation steps and the pass/fail gates.

Deploy the way the house model says — simulate, then enforce

- **Never enforce a web-upload block you have not validated in simulation.** The single most common

cause of a DLP incident is an upload **Block** moved to enforce without a simulation pass — it breaks legitimate work and floods support. The control goes **Audit/simulation** → **validate** → **enforce**, one ring at a time.

- **Small pilot ring first.** Onboard and scope a handful of devices (`Browser & Network DLP Pilot Ring – Stage1`) before the estate. It is far cheaper to find a false positive on ten machines than ten thousand.
- **Escalate the action ladder deliberately.** Start the sensitive service domain group at **Audit**,

move to **Warn (block with override + business justification)**, and only then **Block** the highest-risk destinations. Do not jump straight to Block.

- **Keep a rollback ready.** `Set-DlpCompliancePolicy -Mode TestWithoutNotifications` (back to

simulation) or `-Mode Disable` stops enforcement instantly, and the domain-group action can be returned to **Audit** in the portal — know both before you enforce.

Key the rule on *sensitive* content, not every upload

- **Condition on labels and/or SITs**, not "all content." A policy that flags *every* upload to a

destination generates noise and workarounds; one that flags **labeled/classified** content going to an unsanctioned destination is defensible and proportionate.

- **Lead with sensitivity labels where they exist** — a label is an explicit business decision about

sensitivity and travels with the file. Fall back to **sensitive information types** where labels are not yet adopted, and add label conditions as labeling matures.

- **Tune SIT confidence and count** to reduce false positives before you enforce.

Get the browser & domain settings right — they are the vocabulary

- **Curate the sensitive service domain group deliberately.** `Unsanctioned & Personal Cloud – Domain

Group` is what the upload restriction acts on. Build it from where uploads *actually* go (Activity Explorer, and any proxy / Defender for Cloud Apps discovery), and keep it current as the business adopts new tools. An incomplete list leaves egress paths uncontrolled; an over-broad one blocks sanctioned work.

- **Distinguish sanctioned from unsanctioned destinations.** Corporate OneDrive/SharePoint and approved

SaaS are sanctioned and should not be in the restricted group; personal/consumer cloud and unsanctioned file-sharing are what you control.

- **Cover all three browsers.** Edge is native; **Chrome and Firefox need the Microsoft Purview browser

extension **or their upload activity is invisible. An un-extended browser is an open door — treat it as an** unallowed browser** until the extension is deployed and confirmed.

- **Use the unallowed-browsers control as a steering tool, not a guarantee.** It steers users toward a

browser Purview can see into; it is not a wall against a determined user on unmanaged equipment.

Onboarding & extension hygiene

- **Onboarding is shared with Defender for Endpoint** — coordinate so you do not double-manage or fight

another team's onboarding/offboarding. Confirm devices in **Purview** → **Devices**, not by assumption (`DLPBROWSER-ScopeHealth.ps1` reports a proxy count; the portal is authoritative).

- **Manage the extension through enterprise browser policy** (Group Policy / Intune / browser

management) and verify it is actually installed and enabled on sample devices. Re-check after browser or extension updates — the extension name/ID can change.

- **Keep the endpoint client and OS current** — endpoint sensing depends on it; unpatched devices under-report.

Operate it, don't set-and-forget

- **Review upload/egress activity regularly** (`DLPBROWSER-Report.ps1`, and Activity explorer filtered to Devices). Rising activity to a destination is a signal to review the domain group, tuning, or user education — not automatically a reason to block harder.
- **Watch override / business-justification trends.** A destination that is constantly overridden may be mis-scoped or may indicate a genuine business need that belongs in the sanctioned list, not a block.
- **Version and name policies clearly.** Move from `Browser & Network DLP – Devices – Pilot` to a production name as you exit pilot; keep a change log of domain-group, scope, and action changes.
- **Re-validate after portal changes.** The Purview portal and the Endpoint DLP browser/domain settings evolve; re-run `DLPBROWSER-Validate.ps1` after Microsoft feature updates and confirm nothing regressed.

Keep the roles and determinations where they belong

- **Least privilege, split duties.** DLP authoring and device onboarding are different roles — do not fold them into one over-privileged account. Author with Compliance Administrator / Compliance Data Administrator, not standing Global Administrator.
- **Determinations stay with the owners.** *Which* data is sensitive, *which* destinations are "unsanctioned," and *whether* an upload is a violation are **compliance / security / HR-legal** calls. You provide the configuration and the evidence, not the judgement (see `global-requirements-purview.md` §5).
- **Communicate before you enforce.** Policy tips and user notice reduce friction and support the lawful, transparent operation of a web-upload control; surprise blocks generate tickets and workarounds.

Security & handling of the tooling

- **Read-only / `-WhatIf` by default.** Run the provided scripts in their safe default first; only add `-Commit` when you intend to write, under change control.
- **No secrets in code.** The scripts use interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`); never hardcode credentials, tokens, or app secrets.
- **Least-privilege connections.** Connect with the least-privilege role the task needs, not a standing

Global Administrator session; the assessment/health scripts need only Graph **read** scopes.

Deliverables Summary — Browser & Network DLP in Microsoft Purview

Everything included in this solution, what each piece is for, and how it maps to the deployment model. Use it as the table of contents and the "did we finish?" checklist.

Documents

Document	What it gives you
01 — Solution Overview	What Browser & Network DLP delivers, how the Microsoft Purview browser extension + native Edge + unallowed browsers + sensitive service domains control web uploads, the simulate-first rule, and the Assess→Design→Configure→Pilot→Validate→Expand model.
02 — Requirements	The solution-specific prerequisites (2.1–2.9): Endpoint DLP licensing, least-privilege DLP roles, the S&C PowerShell + Graph toolchain and read scopes, onboarded devices, the browser extension + browser coverage, in-scope labels/SITs, the unsanctioned/personal-cloud destination inventory, and a pilot ring in simulation — each with Verify / Implement-if-missing. References the category `global-requirements-purview.md`.
03 — Implementation Guide	The ordered build: assess tenant + endpoints → design condition/destinations/action ladder → confirm onboarding + extension → build the sensitive service domain group (Audit) + unallowed browsers → author the policy + upload rule **in simulation** → pilot & tune → validate → escalate to Warn/Block + expand. Every object named, with a pass/fail gate and rollback per phase.
05 — Testing & Validation	The pass/fail gates: licensing/availability, onboarding + browser coverage, dry-run makes no changes, policy created in simulation, sensitive upload detected (not over-matching), unallowed-browser steering, evidence captured, and a controlled enforcement transition.
06 — Troubleshooting	The failures you will actually hit — no upload activity at all, Chrome/Firefox invisible, false positives, the rule not firing, simulation-vs-enforce confusion, `New-DlpComplianceRule` errors, empty reports, and emergency rollback — each with cause and fix.

Document	What it gives you
07 — Best Practices	How to run it well: simulate-then-enforce discipline, keying on sensitive content, curating the domain group and browser coverage, onboarding/extension hygiene, ongoing operation, and role/determination boundaries.
Executive Summary	The one-page business-level summary for stakeholders and sign-off.

PowerShell tools (5)

All are **read-only or** `-WhatIf` **by default**, use interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`), and contain **no credentials**. They do **not** onboard devices, deploy the browser extension, or script the portal-managed browser/domain settings, and do **not** invent cmdlets.

Script	Purpose	Safe default
DLPBROWSER-Discovery.ps1	Inventory existing DLP policies/rules and **Devices** coverage, read the Endpoint DLP browser/domain settings (sensitive service domain group + unallowed browsers) via `Get-PolicyConfig`, and list labels/SITs — the baseline of what already controls web uploads.	Read-only
DLPBROWSER-Configure.ps1	Build/ensure the `Browser & Network DLP – Devices – Pilot` policy and the `Restrict Upload to Unsanctioned Domains – Rule` **in simulation**; requires `-Commit` to write and supports `-WhatIf`. Prints the portal-managed domain-group / unallowed-browser steps.	`-WhatIf` (preview)
DLPBROWSER-Validate.ps1	Run the configuration gates (policy exists, Devices location, simulation-first, pilot scope, upload rule + content condition, domain-group/unallowed-browser readable) with PASS/FAIL/UNKNOWN output.	Read-only
DLPBROWSER-Report.ps1	Roll up policy modes and, where available, export **browser/network upload DLP** match & block events from Activity Explorer to CSV — the operational + evidence view.	Read-only

Script	Purpose	Safe default
DLPBROWSER-ScopeHealth.ps1	Report device onboarding, browser-extension, network-path coverage and Endpoint DLP **licensing** readiness flags (Graph SKU + managed-device signals, pilot group, policy scope, checklist).	Read-only

How it maps to the deployment model

Phase	Deliverables used
Assess	01, 02, `DLPBROWSER-Discovery.ps1`, `DLPBROWSER-ScopeHealth.ps1`
Design	02, 03 (named objects + action ladder), 07
Configure	03, `DLPBROWSER-Configure.ps1 -WhatIf` then `-Commit` (simulation); portal-managed domain group + unallowed browsers
Pilot (simulation)	03, 05 (simulation pass), `DLPBROWSER-Report.ps1`
Validate	05, `DLPBROWSER-Validate.ps1`, `DLPBROWSER-Report.ps1`
Expand (enforce)	03 (Step 8), 05 (enforcement pass), 06, 07

Definition of done

• Pilot devices **onboarded** and visible in **Purview** → **Devices**; Microsoft Purview browser extension deployed on Chrome/Firefox and Edge native coverage confirmed.

• Sensitive service domain group `Unsanctioned & Personal Cloud – Domain Group` built (starting action **Audit**) and the unallowed-browsers list set.

• `Browser & Network DLP – Devices – Pilot` with the upload-restriction rule built **in simulation**, all configuration gates PASS.

• Simulation pass complete (sensitive uploads detected, not over-matching, unallowed-browser steering works); false-positive tuning done; then the **enforcement pass** for Warn/Block per the approved design.

• **Evidence pack** (`DLPBROWSER-Report.ps1` export + Activity explorer + validation output) captured and stored with the change record.

• Regulatory / policy determinations (which data, which destinations, which action) recorded by the **compliance / security** owners — not by the administrator.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DLPBROWSER-Configure.ps1

Purpose: Creates (or ensures) a Devices-scoped DLP compliance policy for the "upload to a restricted cloud service domain / access from an unallowed browser" scenario, in SIMULATION/TEST mode, plus one upload-restriction rule set to Audit while in simulation. SAFE BY DEFAULT: previews with -WhatIf and writes NOTHING unless you pass -Commit. It never enforces (never sets Block/Enable) and it does NOT script the Endpoint DLP browser/domain SETTINGS (the sensitive service domain group + unallowed browsers) — those are portal-managed; the script PRINTS what to configure there and keys the rule to a sensitivity label. Moving to enforce is a deliberate step in the implementation guide. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPBROWSER-Configure.ps1 -WhatIf # preview only (default is preview) .\DLPBROWSER-Configure.ps1 -Commit # create policy + rule in SIMULATION .\DLPBROWSER-Configure.ps1 -Commit -PilotGroup "SG-DLPBROWSER-Pilot" -LabelName "Confidential" EXPECTED RESULTS With -Commit: policy 'Browser & Network DLP - Devices - Pilot' exists in a Test/simulation mode, Devices location, pilot scope, with one Audit upload-restriction rule. Validate with DLPBROWSER-Validate.ps1 (expect Gate A PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-DlpCompliancePolicy and New-/Set-DlpComplianceRule; NO invented cmdlets. The endpoint upload-restriction Setting name/value and the simulation Mode value EVOLVE — the script uses documented names and PRINTS them for you to verify (Get-Help New-DlpComplianceRule -Full) against your tenant before committing. It does NOT enforce (no Block/Enable) and does NOT script the sensitive service domain group / unallowed browsers (portal-managed — verify path in your tenant). > [CmdletBinding()] param([switch]$Commit, [switch]$WhatIf, [string]$PolicyName = "Browser & Network DLP - Devices - Pilot", [string]$RuleName = "Restrict Upload to Unsanctioned Domains - Rule", [string]$PilotGroup = "SG-DLPBROWSER-Pilot", # pilot user/security group for the Devices scope [string]$LabelName = "Confidential", # sensitivity label the rule keys on (adjust to your taxonomy) [string]$DomainGroup = "Unsanctioned & Personal Cloud - Domain Group", # portal-managed sensitive service domain group (reference only) [string]$SimMode = "TestWithoutNotifications" # simulation/test mode; verify the exact value in your tenant)

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPBROWSER-Configure.ps1 -WhatIf # preview only (default is preview) .\DLPBROWSER-Configure.ps1 -Commit # create policy + rule in SIMULATION .\DLPBROWSER-Configure.ps1 -Commit -PilotGroup "SG-DLPBROWSER-Pilot" -LabelName "Confidential" EXPECTED RESULTS With -Commit: policy 'Browser & Network DLP - Devices - Pilot' exists in a Test/simulation mode, Devices location, pilot scope, with one Audit upload-restriction rule. Validate with DLPBROWSER-Validate.ps1 (expect Gate A PASS). Without -Commit: a printed plan and no changes. .NOTES White-label Microsoft Purview tooling. -WhatIf/preview by default; -Commit gates all writes. No credentials stored. Uses standard New-/Set-DlpCompliancePolicy and New-/Set-DlpComplianceRule; NO invented cmdlets. The endpoint upload-restriction Setting name/value and the simulation Mode value EVOLVE — the script uses documented names and PRINTS them for you to verify (Get-Help New-DlpComplianceRule -Full) against your tenant before committing. It does NOT enforce (no Block/Enable) and does NOT script the sensitive service domain group / unallowed browsers (portal-managed — verify path in your tenant). > [CmdletBinding()] param([switch]$Commit,

[switch]$WhatIf, [string]$PolicyName = "Browser & Network DLP - Devices - Pilot", [string]$RuleName = "Restrict Upload to Unsanctioned Domains - Rule", [string]$PilotGroup = "SG-DLPBROWSER-Pilot", # pilot user/security group for the Devices scope [string]$LabelName = "Confidential", # sensitivity label the rule keys on (adjust to your taxonomy) [string]$DomainGroup = "Unsanctioned & Personal Cloud - Domain Group", # portal-managed sensitive service domain group (reference only) [string]$SimMode = "TestWithoutNotifications" # simulation/test mode; verify the exact value in your tenant)

DLPBROWSER-Discovery.ps1

Purpose: Read-only baseline for Browser & Network DLP. Inventories the DLP compliance policies/rules already in the tenant and flags which cover the Devices (endpoint) location and their mode (simulation/test vs enable); reads the Endpoint DLP browser/network settings (sensitive service domain groups and the unallowed-browsers list) via Get-PolicyConfig; and lists the sensitivity labels and sensitive information types the upload-restriction rule could key on. This is the "what already controls web uploads" baseline before you build anything. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPBROWSER-Discovery.ps1 .\DLPBROWSER-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS A CSV baseline (.\DLPBROWSER-Discovery.csv) of DLP policies (DevicesLocation flag + Mode), rules, labels, and the endpoint browser/domain settings it could read; with -IncludeSensitiveTypes, SITs to .\DLPBROWSER-SensitiveTypes.csv. A tenant with NO Devices-location policy or NO sensitive service domain group is itself the first finding. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell (Get-DlpCompliancePolicy/-Rule, Get-Label, Get-PolicyConfig); none are invented. Endpoint DLP is licensing-dependent and the browser/domain SETTINGS surface (Get-PolicyConfig shape and the Endpoint DLP settings portal path) EVOLVES — the script WARNS where a value is not readable rather than assuming, and device ONBOARDING is confirmed in the Purview portal (Settings > Device onboarding), which has no supported S&C PowerShell read here. Verify surfaces in your tenant. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\DLPBROWSER-Discovery.csv", [string]$SensitiveOutPath = ".\DLPBROWSER-SensitiveTypes.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPBROWSER-Discovery.ps1 .\DLPBROWSER-Discovery.ps1 -IncludeSensitiveTypes EXPECTED RESULTS A CSV baseline (.\DLPBROWSER-Discovery.csv) of DLP policies (DevicesLocation flag + Mode), rules, labels, and the endpoint browser/domain settings it could read; with -IncludeSensitiveTypes, SITs to .\DLPBROWSER-SensitiveTypes.csv. A tenant with NO Devices-location policy or NO sensitive service domain group is itself the first finding. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Cmdlets are standard Security & Compliance PowerShell (Get-DlpCompliancePolicy/-Rule, Get-Label, Get-PolicyConfig); none are invented. Endpoint DLP is licensing-dependent and the browser/domain SETTINGS surface (Get-PolicyConfig shape and the Endpoint DLP settings portal path) EVOLVES — the script WARNS where a value is not readable rather than assuming, and device ONBOARDING is confirmed in the Purview portal (Settings > Device onboarding), which has no supported S&C PowerShell read here. Verify surfaces in your tenant. > [CmdletBinding()] param([switch]$IncludeSensitiveTypes, [string]$OutPath = ".\DLPBROWSER-Discovery.csv", [string]$SensitiveOutPath = ".\DLPBROWSER-SensitiveTypes.csv")

DLPBROWSER-Report.ps1

Purpose: Read-only operational + evidence report for Browser & Network DLP. Summarizes the DLP policies that cover the Devices location and their mode (simulation vs enforce), and — where the

tenant/licensing exposes it — exports recent browser/network UPLOAD activity from Activity Explorer (cloud-egress / upload-to-restricted-domain / unallowed-browser events) so you can see what the policy would have audited, warned, or blocked. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPBROWSER-Report.ps1 .\DLPBROWSER-Report.ps1 -Days 14 EXPECTED RESULTS A console summary + .\DLPBROWSER-PolicyModes.csv (policies + mode + Devices flag) and, where available, .\DLPBROWSER-UploadActivity.csv (Activity Explorer upload/egress events). If activity export is not available in your tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-DlpCompliancePolicy and Export-ActivityExplorerData (Security & Compliance PowerShell); none invented. Activity Explorer export is licensing-dependent and the endpoint activity-type names EVOLVE — the script filters best-effort on upload/egress/browser keywords and degrades gracefully (warns) rather than assuming availability. This is EVIDENCE for the determination owners, not a verdict. > [CmdletBinding()] param([int]$Days = 7, [string]$ModesOutPath = ".\DLPBROWSER-PolicyModes.csv", [string]$ActivityOutPath = ".\DLPBROWSER-UploadActivity.csv")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPBROWSER-Report.ps1 .\DLPBROWSER-Report.ps1 -Days 14 EXPECTED RESULTS A console summary + .\DLPBROWSER-PolicyModes.csv (policies + mode + Devices flag) and, where available, .\DLPBROWSER-UploadActivity.csv (Activity Explorer upload/egress events). If activity export is not available in your tenant/licence, the script WARNS and reports policy modes only. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses Get-DlpCompliancePolicy and Export-ActivityExplorerData (Security & Compliance PowerShell); none invented. Activity Explorer export is licensing-dependent and the endpoint activity-type names EVOLVE — the script filters best-effort on upload/egress/browser keywords and degrades gracefully (warns) rather than assuming availability. This is EVIDENCE for the determination owners, not a verdict. > [CmdletBinding()] param([int]$Days = 7, [string]$ModesOutPath = ".\DLPBROWSER-PolicyModes.csv", [string]$ActivityOutPath = ".\DLPBROWSER-UploadActivity.csv")

DLPBROWSER-ScopeHealth.ps1

Purpose: Readiness/scope helper for Browser & Network DLP. Reports the Endpoint DLP–bearing licensing signal (subscribedSkus via Graph), managed-device signals (a proxy for onboarding readiness), pilot user/security group membership, the current Devices-location scope of the policy, and a readiness CHECKLIST for the coverage that has no direct PowerShell read — the Microsoft Purview browser extension (Chrome/Firefox), native Edge integration, and any network-DLP path. Read-only; it never changes scope (it prints the command for you to run under change control). HOW TO RUN PowerShell 5.1+ or 7+ with Microsoft Graph and (optionally) Security & Compliance PowerShell: Connect-MgGraph -Scopes "Organization.Read.All","Device.Read.All","Group.Read.All" Connect-IPPSSession # optional, to read the policy scope .\DLPBROWSER-ScopeHealth.ps1 .\DLPBROWSER-ScopeHealth.ps1 -PilotGroup "SG-DLPBROWSER-Pilot" EXPECTED RESULTS A console summary: Endpoint DLP licensing flag, managed-device count (Graph), pilot-group membership, the policy's current Devices scope, and a printed extension/Edge/network-path readiness checklist. Confirm actual device ONBOARDING and EXTENSION deployment in the Purview portal / your MDM (no PowerShell equivalent). .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses standard Microsoft Graph (Get-MgSubscribedSku / Get-MgDeviceManagementManagedDevice / Get-MgGroup) and Get-DlpCompliancePolicy; none invented. Graph managed-device data and the SKU flag are PROXIES/readiness signals — Endpoint DLP onboarding, browser-extension presence, and network-DLP coverage are confirmed in the Purview portal /

your MDM, and the SKU-to-capability mapping is licensing-dependent (verify against your plan). > [CmdletBinding()] param([string]$PilotGroup = "SG-DLPBROWSER-Pilot", [string]$PolicyName = "Browser & Network DLP - Devices - Pilot")

How to run: PowerShell 5.1+ or 7+ with Microsoft Graph and (optionally) Security & Compliance PowerShell: Connect-MgGraph -Scopes "Organization.Read.All","Device.Read.All","Group.Read.All" Connect-IPPSSession # optional, to read the policy scope .\DLPBROWSER-ScopeHealth.ps1 .\DLPBROWSER-ScopeHealth.ps1 -PilotGroup "SG-DLPBROWSER-Pilot" EXPECTED RESULTS A console summary: Endpoint DLP licensing flag, managed-device count (Graph), pilot-group membership, the policy's current Devices scope, and a printed extension/Edge/network-path readiness checklist. Confirm actual device ONBOARDING and EXTENSION deployment in the Purview portal / your MDM (no PowerShell equivalent). .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Uses standard Microsoft Graph (Get-MgSubscribedSku / Get-MgDeviceManagementManagedDevice / Get-MgGroup) and Get-DlpCompliancePolicy; none invented. Graph managed-device data and the SKU flag are PROXIES/readiness signals — Endpoint DLP onboarding, browser-extension presence, and network-DLP coverage are confirmed in the Purview portal / your MDM, and the SKU-to-capability mapping is licensing-dependent (verify against your plan). > [CmdletBinding()] param([string]$PilotGroup = "SG-DLPBROWSER-Pilot", [string]$PolicyName = "Browser & Network DLP - Devices - Pilot")

DLPBROWSER-Validate.ps1

Purpose: Read-only validation of the Browser & Network DLP policy against the configuration gates in 05-testing.md (Gate A). Confirms the policy exists, covers the Devices location, is in a Test/simulation mode (during the pilot pass), is scoped to a pilot (not all devices), and carries the upload-restriction rule with a sensitive-content condition; and reads the Endpoint DLP browser/domain SETTINGS (sensitive service domain group + unallowed browsers) via Get-PolicyConfig to confirm the upload restriction has somewhere to act. Prints PASS/FAIL per check. Changes nothing. HOW TO RUN PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPBROWSER-Validate.ps1 .\DLPBROWSER-Validate.ps1 -PolicyName "Browser & Network DLP - Devices - Pilot" EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, A3 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard Get-DlpCompliance* and Get-PolicyConfig cmdlets; none invented. The browser/domain SETTINGS surface EVOLVES — where the domain group / unallowed browsers are not readable via PowerShell, the gate reports UNKNOWN and asks you to confirm in the portal rather than failing. Whether the policy SHOULD be in simulation depends on the phase — the script reports the mode and lets you judge against the phase you are in. > [CmdletBinding()] param([string]$PolicyName = "Browser & Network DLP - Devices - Pilot", [string]$RuleName = "Restrict Upload to Unsanctioned Domains - Rule")

How to run: PowerShell 5.1+ or 7+ with Security & Compliance PowerShell (Connect-IPPSSession): Connect-IPPSSession .\DLPBROWSER-Validate.ps1 .\DLPBROWSER-Validate.ps1 -PolicyName "Browser & Network DLP - Devices - Pilot" EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Expect all-PASS in the simulation pass. In the enforce pass, A3 (simulation) is expected to FLIP (policy Mode = Enable) BY DESIGN — that is noted, not an error. .NOTES White-label Microsoft Purview tooling. Read-only. No credentials stored. Standard Get-DlpCompliance* and Get-PolicyConfig cmdlets; none invented. The browser/domain SETTINGS surface EVOLVES — where the domain group / unallowed browsers are not readable via PowerShell, the gate reports UNKNOWN and asks you to confirm in the portal rather than failing. Whether the policy SHOULD be in simulation depends on the phase — the

script reports the mode and lets you judge against the phase you are in. > [CmdletBinding()] param([string]$PolicyName = "Browser & Network DLP - Devices - Pilot", [string]$RuleName = "Restrict Upload to Unsanctioned Domains - Rule")